Smart Traffic Management System

7-Level System Architecture, OS & DBMS Module Mapping, and Viva Guide

Team StructMinds (Ayush Gaur, Vedansh Manwal, Anmol Pujari)

1. Approach Verification: 100% Match to 7-Level Pipeline

Our implementation follows the exact hierarchical model designed for this project. The simulation flows from environment sensing down to decision arbitration, actuation, and persistent database logging.

Level	Module Name	Exact File Location	Technical Responsibility
Level 1	Simulation Environment	TrafficControl/network.net.xml TrafficControl/routes.rou.xml	Builds microscopic 4-way left-hand intersection and vehicle routes in SUMO.
Level 2	Main Controller	src/simulation/sumo_env.py main.py / gui_app.py	Python TraCI bridge extracting vehicle counts, speeds, waiting times, and queues.
Level 3A	AI Core Module (DQN)	src/rl/dqn_agent.py src/rl/train_dqn.py	PyTorch Deep Q-Network observing states and recommending green phase actions.
Level 3B	OS Layer (Scheduler)	src/scheduling/scheduler.py	Real-time Dynamic Priority scheduling, Aging, and anti-starvation counters.
Level 4	Decision & Safety Layer	src/simulation/signal_controller.py scheduler.py (arbitrate)	Guarantees safe clearance: min green (10s), max green (60s), 3s yellow, OS veto.
Level 5	Action Implementation	src/simulation/sumo_env.py	Applies validated signal states directly to junction C in SUMO via TraCI.
Level 6	Data Persistence (DBMS)	src/database/db_manager.py	Stores runs, metrics, and decisions in MySQL (with auto SQLite fallback).
Level 7	Analysis & Metrics	src/evaluation/evaluator.py src/evaluation/plot_results.py	Computes throughput, delay, max queue, and exports comparison charts.

2. Where Operating Systems (OS) Concepts Are Implemented

Primary Code File: src/scheduling/scheduler.py

- **Ready Processes:** Directional traffic approaches (NS and EW flows) are instances of *TrafficProcess* with dynamic queues and state.
- **CPU Time Slice:** Signal green time is treated as CPU execution quantum constrained by min green (10s) and max green (60s).
- **Dynamic Priority:** *Priority = (Queue Length * w_q) + (Starvation Time * w_a)*. Bounded waiting is maintained dynamically.
- **Process Aging:** Time spent waiting without green light increments *time_starved* every second, boosting priority to prevent starvation.
- **Kernel Arbiter / Veto:** The *arbitrate()* function inspects the AI agent's proposals. If a direction exceeds max starvation (45s), the OS layer vetoes the AI.
- **Context Switch Cost:** Mandatory 3-second yellow clearance represents context-switch overhead, preventing high-frequency thrashing.

3. Where Database Management (DBMS) Is Implemented

Primary Code File: src/database/db_manager.py | **Storage:** results/traffic_results.sqlite3

- **Normalized Relational Schema:** Three distinct relational tables with primary and foreign keys:

- *simulation_runs*: Summary metrics (run_id PK, controller, arrived, stopped_veh_sec, CO2, throughput).

- *step_metrics*: Second-by-second traffic observations (run_id FK, sim_time, active_vehicles, halted, queues).

- *scheduler_decisions*: OS & AI arbitration audit logs (run_id FK, chosen_phase, reason, waiting times).

• **ACID Transactions & Parameterized Queries:** Parameterized SQL queries prevent injection and ensure atomic *commit()* / *rollback()*.

• **Dual Driver Architecture:** Production-ready MySQL connection with automated zero-crash fallback to SQLite for local presentations.

4. Clean Project Directory Structure

The directory has been cleaned to keep strictly the essential, demonstrable modules:

• **run_dashboard.bat & gui_app.py:** Desktop visual dashboard with live terminal logs, seed selector, and auto-zoomed SUMO visualizer.

• **main.py:** Unified CLI entry point for scripted benchmarks, single runs, and training.

• **TrafficControl/:** SUMO simulation network files, route demand, and viewsettings.xml (auto-zoom + real-world scheme).

• **src/scheduling/:** OS Process Scheduling (TrafficProcess, Priority, Aging, Arbitration).

• **src/rl/:** Deep Q-Network agent (PyTorch policy and target networks, experience replay).

• **src/simulation/:** TraCI Gym-like environment and SignalController safety interval manager.

• **src/database/:** MySQL / SQLite persistence and transactional storage.

• **src/evaluation/:** 4-controller benchmarking engine and comparison chart plotter.

• **tests/:** 10 automated unit tests (test_traffic_system.py) verifying safety, OS, AI, and DBMS.

• **results/:** benchmark_comparison.png and traffic_results.sqlite3.

• **ProjectInfo/:** This architecture documentation and viva reference guide.